MODULE 03 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

# Arithmetic operators and precedence

Seven operators, which ones give floats, what happens with negatives, and the order Python works in.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 03 of 28

admin@sthithapragna.com

Every arithmetic answer turns on three questions: **which operator goes first**, which direction it groups, and whether the result is an int or a float.

# Arithmetic operators at a glance

| OPERATOR | NAME | 7 OP 2 | RESULT TYPE WITH TWO INTS |
|---|---|---|---|
| `+ - *` | add, subtract, multiply | 9 5 14 | int |
| `/` | true division | 3.5 | always float |
| `//` | floor division | 3 | int |
| `%` | remainder, or modulo | 1 | int |
| `**` | power, or exponent | 49 | int, unless the exponent is negative |

THE TRAP

/ always returns a float, even when the division is exact. 6 / 3 is 2.0.

# The rule for the result type

```
1  print(6 / 3)       # 2.0   / always gives a float
2  print(6 // 3)      # 2     int // int gives an int
3  print(4.5 // 2)    # 2.0   one float operand makes a float
4  print(1 + 1.0)     # 2.0
5  print(2 ** -1)     # 0.5   negative exponent gives a float
6  print(True + 2)    # 3     True counts as 1
```

1 Single slash is the one operator that always returns a float.

3-4 For the others: if either operand is a float, the result is a float.

# // rounds down, towards minus infinity

```
1  print(7 // 2)      # 3     3.5 rounds down to 3
2  print(-7 // 2)     # -4    -3.5 rounds down to -4
3  print(7 // -2)     # -4
4  print(-7 // -2)    # 3     3.5 rounds down to 3
```

1 Floor means the next whole number to the left on the number line.

2-3 For negatives, left is further from zero. −3.5 floors to −4, not −3.

Division by zero with /, // or % raises ZeroDivisionError.

# % takes the sign of the divisor

```
1  print(7 % 3)       # 1
2  print(-7 % 3)      # 2     sign follows the 3
3  print(7 % -3)      # -2    sign follows the -3
4  print(6.5 % 2)     # 0.5   works on floats too
5  print(10 % 2 == 0) # True  the classic even-number test
```

2 Check: -7 // 3 is -3, and -3 * 3 + 2 is -7. The remainder fills the gap.

5 n % 2 == 0 is how Python code tests for an even number.

# ** groups right to left, and beats unary minus

```
1  print(2 ** 3 ** 2)     # 512   means 2 ** (3 ** 2)
2  print((2 ** 3) ** 2)   # 64
3  print(-2 ** 2)         # -4    means -(2 ** 2)
4  print((-2) ** 2)       # 4
5  print(2 ** -2)         # 0.25  minus on the right belongs to 2
6  print(4 ** 0.5)        # 2.0   a fractional power is a root
```

1 Right-sided binding: the rightmost ** is done first.

3 ** is stronger than a minus on its left. The minus is applied last.

# Arithmetic precedence, highest first

| OPERATORS | GROUPS |
| --- | --- |
| `( ) brackets` | innermost first |
| `**` | right to left |
| `+x -x unary plus and minus` | right to left |
| `* / // %` | left to right |
| `+ - binary add and subtract` | left to right |
| `= += -= and other assignments` | done last |

THE TRAP

* / // and % share one level. Between them, go left to right. Multiplication does not beat division.

# When priorities tie, read left to right

```
1  print(12 / 2 * 3)     # 18.0   (12 / 2) * 3, not 12 / 6
2  print(1 // 2 * 3)     # 0      (1 // 2) is 0, times 3 is 0
3  print(8 // 3 % 2)     # 0      (8 // 3) is 2, 2 % 2 is 0
4  print(100 - 10 - 5)   # 85     (100 - 10) - 5
```

1. Twelve divided by two is six, times three is eighteen. And / made it a float.
2. Floor division first gives 0, so the multiplication cannot rescue it.

# 2 + 3 * 4 ** 2 // 5 - 1, one step at a time

01

### Power first

4 ** 2 is 16. Now: 2 + 3 * 16 // 5 - 1

02

### * and //, left to right

3 * 16 is 48, then 48 // 5 is 9. Now: 2 + 9 - 1

03

### + and -, left to right

2 + 9 is 11, then 11 - 1 is 10.

04

### Result

10, an int. No / anywhere, no float operand.

EXAM TECHNIQUE

Take every step, in order, even when the expression looks easy. Skipping steps is where precedence mistakes happen.

# Signs on their own

```
1  x = 1
2  print(+++x)          # 1     three pluses change nothing
3  print(- - -1)        # -1    count the minuses: odd is negative
4  print(2 - -3)        # 5     binary minus, then unary minus
5  print(3 * -2)        # -6
```

2 Unary plus leaves a number unchanged, however many you stack.

3 Each unary minus flips the sign. An odd count is negative.

There is no increment operator. x++ is a SyntaxError, and ++x is just +(+x): x does not change.

# Augmented assignment: the right side goes first

```
1  x = 10
2  x += 5          # x = x + 5       -> 15
3  x //= 4         # x = x // 4      -> 3
4  x **= 2         # x = x ** 2      -> 9
5  y = 4
6  y *= 2 + 3      # y = y * (2 + 3) -> 20, not 11
```

2-4 x op= value means x = x op value. Every arithmetic operator has one.

6 The whole right side is worked out first, as if it were in brackets.

# + joins strings, * repeats them

```
1  print("ab" + "cd")       # abcd
2  print("ab" * 3)          # abab
3  print(3 * "ab" + "x")    # abababx   * before +, as usual
4  print("ab" * 0)          # an empty line: ""
5  print("ab" + 3)          # TypeError
```

1-3 Plus joins, times repeats, and precedence works exactly as for numbers.

5 You cannot add a str and an int. Python never converts for you.

Only + and * work on strings. Subtracting or dividing strings is a TypeError. += and *= work too.

# Why 0.1 + 0.2 is not 0.3

```
1  print(0.1 + 0.2)                  # 0.30000000000000004
2  print(0.1 + 0.2 == 0.3)           # False
3  print(round(0.1 + 0.2, 2) == 0.3) # True
4  print(1 / 3)                      # 0.3333333333333333
```

1. Floats are stored in binary. Most decimal fractions cannot be stored exactly.
2. So never compare floats for exact equality. Round them, or check they are close.

# round() sends exact halves to the even neighbour

```
1  print(round(2.5))        # 2     not 3
2  print(round(3.5))        # 4
3  print(round(7.456, 2))   # 7.46
4  print(round(3.7))        # 4     an int
5  print(round(3.7, 0))     # 4.0   a float, because of the second argument
6  print(round(2.675, 2))   # 2.67  2.675 is stored as slightly less
```

1-2 An exact half rounds to the nearest even number. This avoids bias across many values.

6 Floating-point accuracy again: the stored value is just below the half.

# Error messages you may see, and what they mean

- Python 3.12: 5 % 0 says **integer modulo by zero**; 5 // 0 says **integer division or modulo by zero**.
- Python 3.14: all of these now just say **division by zero**.
- The exception is **ZeroDivisionError** in every version. That is what the exam asks about.

> ON THE EXAM
>
> Answer by the exception name, never by the exact message text. Messages change between versions.

# What to carry into the exam

- **/ always gives a float.** Otherwise, any float operand gives a float.
- **//** rounds down: -7 // 2 is -4. **%** takes the divisor's sign: -7 % 3 is 2.
- ****** groups right to left, and -2 ** 2 is -4.
- * / // % share a level: left to right. In x *= a + b, a + b goes first.
- 0.1 + 0.2 == 0.3 is False. round(2.5) is 2.

## What is the output of this program?

```
1  x = 1 // 5 + 1 / 5
2  print(x)
```

A 0.2

B 0.4

C 0

D 0.0

◆ PAUSE AND DECIDE

## What is the output of this program?

```
1  x = 1 // 5 + 1 / 5
2  print(x)
```

1 // 5 is 0. 1 / 5 is 0.2. 0 + 0.2 is 0.2

A `0.2`

Floor division gives 0, true division gives 0.2, and 0 + 0.2 is 0.2.

B `0.4`

*Treats both divisions as true division.*

C `0`

*Treats both as floor division. The / makes a float anyway.*

D `0.0`

*Rounds the 0.2 away. Nothing here discards the fraction of 1 / 5.*

THE SIGNAL

Evaluate each division separately, with its own rule, before you add.

## What is the output of this program?

```
1  print(2 ** 3 ** 2 ** 1)
```

A 64

B 512.0

C 512

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

## What is the output of this program?

```
1  print(2 ** 3 ** 2 ** 1)
```

2 ** 1 is 2. 3 ** 2 is 9. 2 ** 9 is 512.

A 64

*Groups left to right: 8 ** 2 ** 1. Power groups right to left.*

B 512.0

*Right value, wrong type. All ints and no negative exponent: an int.*

C 512

Right to left: 2 ** 1, then 3 ** 2, then 2 ** 9.

D The code raises a SyntaxError

*Chained powers are perfectly legal.*

THE SIGNAL

** is the one arithmetic operator that groups right to left. Start at the far right.

## What is the output of this program?

```
1  a = -7 // 2
2  b = -7 % 2
3  print(a, b)
```

A  -4 1

B  -3 -1

C  -3 1

D  -4 -1

◆ PAUSE AND DECIDE

## What is the output of this program?

```
1  a = -7 // 2
2  b = -7 % 2
3  print(a, b)
```

Check: -4 * 2 + 1 is -7.

A `-4 1`

-3.5 floors to -4. The remainder takes the sign of 2.

B `-3 -1`

*Chops towards zero, the way C and Java do. Python floors.*

C `-3 1`

*Right remainder, but -3.5 does not floor to -3.*

D `-4 -1`

*Right floor, but the remainder follows the divisor, which is positive.*

THE SIGNAL

// rounds towards minus infinity, and % takes the sign of the right-hand number.

## What is the output of this program?

```
1  x = 4
2  x *= 2 + 3
3  x //= 3
4  print(x)
```

A 3

B 6.666666666666667

C 6.0

D 6

◆ PAUSE AND DECIDE

## What is the output of this program?

```
1  x = 4
2  x *= 2 + 3
3  x //= 3
4  print(x)
```

x = 4 * (2 + 3) = 20, then x = 20 // 3 = 6

A 3

*Reads line 2 as x * 2 + 3, which is 11. The right side goes first.*

B 6.666666666666667

*That would be /=. Floor division drops the fraction.*

C 6.0

*Every value is an int, so // gives an int.*

D 6

4 * (2 + 3) is 20, then 20 // 3 is 6.

THE SIGNAL

x op= expression means x = x op (expression). The brackets are invisible, but always there.

## What is the output of this program?

```
1  print(-2 ** 2 + 10 // 4 * 2)
```

A 0

B 8

C 5.0

D -8

◆ PAUSE AND DECIDE

## What is the output of this program?

```
1  print(-2 ** 2 + 10 // 4 * 2)
```

-(2 ** 2) is -4. 10 // 4 is 2, times 2 is 4. -4 + 4 is 0.

A `0`

The square happens before the minus: -4. Then 10 // 4 * 2 is 4.

B `8`

*Squares -2 to get 4. But ** beats the minus on its left.*

C `5.0`

*Treats // as / and 4 * 2 as first. Neither is right.*

D `-8`

*Multiplies 4 * 2 before the floor division. They share a level: left to right.*

THE SIGNAL

A minus on the left of ** is applied after the power. A minus on the right belongs to the exponent.

## What is the output of this program?

```
1  print(3 * "ab" + "c" * 2)
```

A `abababcababababc`

B `abababcc`

C The code raises a TypeError

D `ab ab ab cc`

◆ PAUSE AND DECIDE

## What is the output of this program?

```
1  print(3 * "ab" + "c" * 2)
```

3 * "ab" is "ababab". "c" * 2 is "cc". Joined: "abababcc"

A `abababcabababc`

*Works left to right, ignoring precedence: ((3 * ab) + c) * 2.*

B `abababcc`

Both repetitions happen first, then + joins them.

C The code raises a TypeError

*int * str is legal. Only str + int fails.*

D `ab ab ab cc`

*Repetition adds no separators. That is print's sep, not *.*

THE SIGNAL

Precedence is the same for strings as for numbers: * before +.

## What is the output of this program?

```
1  total = 0.1 + 0.2
2  print(total == 0.3, round(total, 1) == 0.3)
```

A True True

B False True

C False False

D True False

◆ PAUSE AND DECIDE

## What is the output of this program?

```
1  total = 0.1 + 0.2
2  print(total == 0.3, round(total, 1) == 0.3)
```

total is 0.30000000000000004

**A** `True True`

*Assumes floats are exact. 0.1 and 0.2 are stored approximately.*

**B** `False True`

The tiny error breaks ==. Rounding to 1 place removes it.

**C** `False False`

*round(total, 1) is 0.3, the same float as the literal 0.3.*

**D** `True False`

*Reverses the effect. Rounding can only help the comparison here.*

THE SIGNAL

Never test floats with == directly. Round both sides, or check that they are close.

## What is the output of this program?

```
1  x = 7 / 7
2  y = 7 // 7.0
3  z = 7 % 3
4  print(x, y, z)
```

A `1 1 1`

B `1.0 1 1`

C `1.0 1.0 1`

D `1.0 1.0 1.0`

◆ PAUSE AND DECIDE

## What is the output of this program?

```
1  x = 7 / 7
2  y = 7 // 7.0
3  z = 7 % 3
4  print(x, y, z)
```

A `1 1 1`

*Forgets that / always returns a float.*

B `1.0 1 1`

*Forgets that a float operand makes // return a float.*

C **`1.0 1.0 1`**

/ gives a float; 7.0 makes // a float; 7 % 3 is all ints.

D `1.0 1.0 1.0`

*Over-applies the float rule. z has no float anywhere.*

THE SIGNAL

Decide the type of each result on its own: / always float, others float only if an operand is.